🧪 Intern Case Study: E-commerce Analytics Pipeline

Duration: 2 Days | **Format:** Individual Submission

📌 Overview

You are joining the data team of **ShopStream**, a mid-sized e-commerce platform. Your task is to take a provided dataset, build a data pipeline into a relational database of your choice, expose it through a Cube.js semantic layer, and build a simple analytics dashboard on top.

📦 Dataset

You are provided with a clean, ready-to-use dataset of **5 CSV files**. No cleaning is required — focus your energy on the pipeline, modeling, and visualization.

Run the script below to generate your dataset:

```python
```

```python
# generate_dataset.py
# Install dependencies: pip install faker pandas numpy

import pandas as pd
import numpy as np
import random
from faker import Faker
from datetime import datetime, timedelta

fake = Faker()
np.random.seed(42)
random.seed(42)

N_CUSTOMERS = 3000
N_PRODUCTS = 300
N_ORDERS = 8000

# --- Customers ---
customers = pd.DataFrame({
    'customer_id': [f'C{str(i).zfill(5)}' for i in range(N_CUSTOMERS)],
    'name': [fake.name() for _ in range(N_CUSTOMERS)],
    'email': [fake.email() for _ in range(N_CUSTOMERS)],
    'gender': [random.choice(['male', 'female', 'other']) for _ in range(N_CUSTOMERS)],
    'city': [fake.city() for _ in range(N_CUSTOMERS)],
    'country': [random.choice(['US', 'CA', 'GB', 'AU', 'IN', 'DE']) for _ in range(N_CUSTOMERS)],
    'signup_date': [fake.date_between(start_date='-4y', end_date='-6m') for _ in range(N_CUSTOMERS)],
    'age': [random.randint(18, 70) for _ in range(N_CUSTOMERS)],
    'tier': [random.choice(['bronze', 'silver', 'gold', 'platinum']) for _ in range(N_CUSTOMERS)],
})
customers.to_csv('customers.csv', index=False)

# --- Products ---
categories = ['Electronics', 'Clothing', 'Home & Kitchen', 'Books', 'Sports', 'Beauty', 'Toys']
products = pd.DataFrame({
    'product_id': [f'P{str(i).zfill(4)}' for i in range(N_PRODUCTS)],
    'name': [fake.catch_phrase() for _ in range(N_PRODUCTS)],
    'category': [random.choice(categories) for _ in range(N_PRODUCTS)],
    'sub_category': [fake.word().capitalize() for _ in range(N_PRODUCTS)],
    'price': [round(random.uniform(5, 500), 2) for _ in range(N_PRODUCTS)],
    'cost': [round(random.uniform(2, 250), 2) for _ in range(N_PRODUCTS)],
    'stock_qty': [random.randint(10, 1000) for _ in range(N_PRODUCTS)],
    'is_active': [random.choice([True, False]) for _ in range(N_PRODUCTS)],
})
```

```
products.to_csv('products.csv', index=False)

# --- Orders ---
customer_ids = customers['customer_id'].tolist()
start_date = datetime(2022, 1, 1)

orders = pd.DataFrame({
    'order_id': [f'ORD{str(i).zfill(6)}' for i in range(N_ORDERS)],
    'customer_id': [random.choice(customer_ids) for _ in range(N_ORDERS)],
    'order_date': [start_date + timedelta(days=random.randint(0, 1095)) for _ in range(N_ORDERS)],
    'status': [random.choice(['completed', 'cancelled', 'pending', 'refunded']) for _ in range(N_ORDERS)],
    'channel': [random.choice(['web', 'mobile', 'store']) for _ in range(N_ORDERS)],
    'discount_pct': [round(random.uniform(0, 0.4), 2) for _ in range(N_ORDERS)],
    'shipping_country': [random.choice(['US', 'CA', 'GB', 'AU', 'IN', 'DE']) for _ in range(N_ORDERS)],
})
orders.to_csv('orders.csv', index=False)

# --- Order Items ---
product_ids = products['product_id'].tolist()
order_ids = orders['order_id'].tolist()

items = []
for oid in order_ids:
    for _ in range(random.randint(1, 5)):
        pid = random.choice(product_ids)
        qty = random.randint(1, 8)
        price = round(random.uniform(5, 500), 2)
        items.append({
            'item_id': f'ITEM{str(len(items)).zfill(6)}',
            'order_id': oid,
            'product_id': pid,
            'quantity': qty,
            'unit_price': price,
            'returned': random.choice([True, False]),
        })
pd.DataFrame(items).to_csv('order_items.csv', index=False)

# --- Events ---
event_types = ['page_view', 'add_to_cart', 'checkout_start', 'purchase', 'search', 'product_view']
events = pd.DataFrame({
    'event_id': [f'E{str(i).zfill(7)}' for i in range(50000)],
    'session_id': [fake.uuid4() for _ in range(50000)],
    'customer_id': [random.choice(customer_ids + [None] * 500) for _ in range(50000)],
    'event_type': [random.choice(event_types) for _ in range(50000)],
```

```
    'product_id': [random.choice(product_ids + [None] * 200) for _ in range(50000)],
    'timestamp': [start_date + timedelta(seconds=random.randint(0, 94608000)) for _ in range(50000)],
    'device': [random.choice(['desktop', 'mobile', 'tablet']) for _ in range(50000)],
})
events.to_csv('events.csv', index=False)

print("✅ Dataset generated. 5 CSV files ready.")
```

Files Generated

File	Rows	Description
`customers.csv`	3,000	Customer profiles
`products.csv`	300	Product catalog
`orders.csv`	8,000	Orders placed
`order_items.csv`	~24,000	Line items per order
`events.csv`	50,000	Clickstream events

🗓️ Tasks

Task 1 — Data Pipeline & Database

Write a Python pipeline that loads the CSV files into a **relational database of your choice** (PostgreSQL, MySQL, SQLite, etc.).

Your pipeline should:

- Load all 5 CSVs into the database
- Design a schema that makes sense for analytical queries — think about how the tables relate to each other
- Be runnable with a single command

There are no strict rules on how you structure this — we want to see how you think about data modeling.

Deliverables:

- `pipeline/` folder with your Python scripts

- `schema.sql` or equivalent DDL
- Instructions in your README to run the pipeline

Task 2 — Cube.js Semantic Layer

Set up a Cube.js project (free, open-source) connected to your database and define cubes that expose meaningful metrics for the business.

At minimum, cover:

- Orders (revenue, count, status breakdown)
- Products (sales by category)
- Customers (by country, tier)
- Events (funnel activity)

Use whatever measures, dimensions, and joins make sense to you.

Deliverables:

- `cubejs/schema/` with your cube definition files
- Cube dev server running on `localhost:4000`

Task 3 — Analytics Dashboard

Build a simple dashboard in React that queries the Cube.js API and visualizes the data. Use **Recharts or Chart.js** (both free).

Include **at least 4 charts** of your choosing — pick whatever you think tells the most interesting story about the data. Also include at least one filter (e.g., date range, country, or category) that updates the charts.

Deliverables:

- `dashboard/` React app
- Runs locally with `npm start`

📋 Submission

- Share a **private GitHub repository** and invite your reviewer

- Include a `README.md` with setup instructions for each layer
- Deadline: **to be communicated separately**

⚠️ Rules

- Use only **open-source** tools and libraries
- You may reference documentation and online resources freely
- Be prepared to walk through and explain any part of your code
- If you get stuck, document what you tried — partial submissions are accepted

---

Focus on clarity and good decisions over completeness. We care more about how you think than how much you ship.